

**FULL STACK DEVELOPMENT WITH MERN**

**Project Documentation**

# **BOOK A DOCTOR**

**Doctor Booking Web Application**

*A Full-Stack MERN Application for Patients, Doctors and Administrators*

## Table of Contents

---

1. Introduction
2. Project Overview
3. Architecture
4. Data Model
5. Setup Instructions
6. Folder Structure
7. Running the Application
8. API Documentation
9. Authentication & Authorization
10. User Interface
11. Testing
12. Screenshots or Demo
13. Known Issues
14. Future Enhancements

## 1. Introduction

---

### 1.1 Project Title

Book A Doctor – Doctor Appointment Web Application.

### 1.2 Team Members

**Ankitha Komati**

**Arugula Abhishek**

**Lokesh Kumar Chitrapu**

### 1.3 Document Purpose

This document describes the design, structure, setup process, and functionality of the Book A Doctor application. It is intended for developers who will maintain or extend the codebase, for reviewers evaluating the project, and for anyone who wants to deploy or run the application locally. It captures the architecture, database schema, REST API contract, authentication mechanism, and the user-facing feature set of the platform.

## 2. Project Overview

---

### 2.1 Purpose

Booking a doctor's through phone calls or in-person visits is often slow and inconvenient for both patients and healthcare providers. Prescripto addresses this by offering a centralized digital platform where patients can discover doctors by specialty, check availability, and book a time slot in a few clicks. Doctors gain a dashboard to manage their schedule and track earnings, while administrators can onboard new doctors and supervise the entire appointment pipeline. By combining appointment scheduling with integrated online payments, the platform removes several manual steps from the traditional healthcare booking process.

### 2.2 Problem Statement

Traditional appointment booking relies on phone calls, walk-ins, or fragmented systems that are not accessible outside clinic hours and do not give patients visibility into a doctor's real-time availability. Clinics and individual practitioners also lack a lightweight, affordable system to manage bookings, payments, and patient records in one place. Book A Doctor is designed as a lightweight, self-hostable answer to this gap.

### 2.3 Target Users

| User Type | Description                                                 | Primary Goals                                                                    |
|-----------|-------------------------------------------------------------|----------------------------------------------------------------------------------|
| Patient   | Any individual seeking to book a consultation with a doctor | Find a suitable doctor, book/manage appointments, pay online, maintain a profile |
| Doctor    | A registered healthcare provider on the platform            | Manage incoming appointments, track earnings, keep profile/availability current  |
| Admin     | Platform administrator / clinic operator                    | Onboard doctors, oversee all appointments, monitor platform activity             |

### 2.4 Key Features

#### 2.4.1 Authentication & Roles

- Three-level authentication system: Patient, Doctor, and Admin, each with its own login and dashboard
- JWT-based session handling with role-specific route protection

#### 2.4.2 Patient-Facing Features

- Browse and search doctors by specialty
- View a doctor's profile: qualification, experience, fees, and description
- Book, reschedule awareness, and cancel appointments
- Pay via Cash, Stripe, or Razorpay

- Editable profile with picture, address, gender, and date of birth
- View appointment history (upcoming and past)

#### **2.4.3 Doctor-Facing Features**

- Dashboard showing total earnings, patient count, and appointment count
- List of latest bookings
- Mark appointments as completed or cancel them
- Update profile: description, consultation fee, address, and availability toggle

#### **2.4.4 Admin-Facing Features**

- Add new doctors with image, specialty, degree, experience, fees, and address
- View and manage the full list of registered doctors
- Dashboard analytics: total doctors, total appointments, total patients, recent bookings
- Cancel or complete any appointment across the platform

### **2.5 Non-Functional Characteristics**

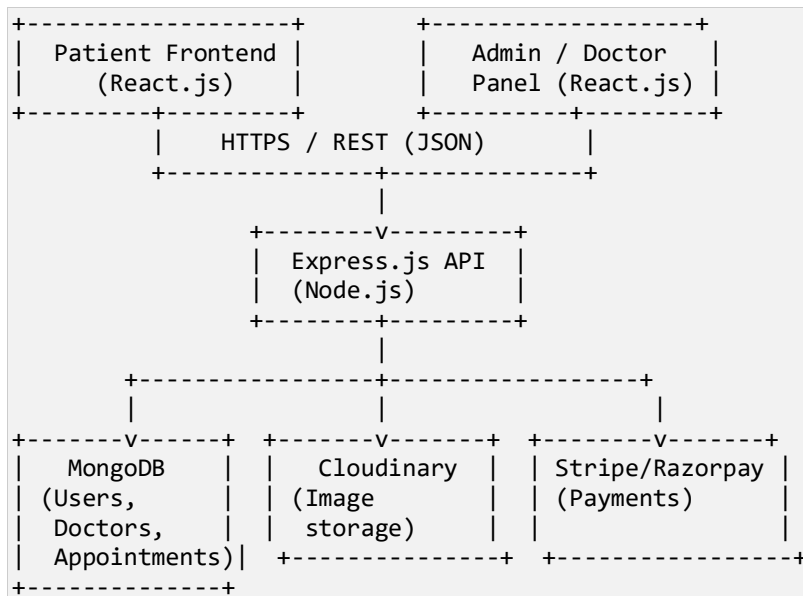
| Attribute       | Approach                                                                                     |
|-----------------|----------------------------------------------------------------------------------------------|
| Security        | JWT-based authentication, hashed passwords, role-gated middleware                            |
| Scalability     | Stateless REST API and a document database (MongoDB) that scale horizontally                 |
| Usability       | Responsive layout built with React and Tailwind-style utility CSS for mobile and desktop     |
| Maintainability | Clear separation between models, controllers, routes, and middleware on the backend          |
| Portability     | Deployable to any Node-compatible host; frontend deployable as a static build (e.g., Vercel) |

## 3. Architecture

### 3.1 High-Level Architecture

This follows a three-tier client-server architecture typical of MERN applications. Three independent client applications (the patient-facing site and the admin/doctor panel) communicate with a single Express.js REST API over HTTPS. The API, in turn, communicates with a MongoDB database for persistence and with third-party services (Cloudinary for image storage, Stripe and Razorpay for payments) for supporting functionality.

Figure 3.1 – Conceptual overview of the three-tier architecture



### 3.2 Frontend Architecture

The patient-facing frontend is a single-page application built with React.js and client-side routing (React Router). It is organized around reusable presentational components (navbar, footer, doctor card, related-doctors carousel) and page-level containers that fetch data from the API and render the appropriate view. Global state, such as the logged-in user's authentication token and the cached list of doctors, is shared through React Context so that it does not need to be re-fetched on every page transition. Axios (or the Fetch API) is used to call the backend, and a token stored client-side is attached to authenticated requests.

The admin panel is a separate React application with its own routing and layout, intentionally decoupled from the patient site so that administrator and doctor tooling can evolve independently and is not bundled into the (larger) public-facing build.

### 3.3 Backend Architecture

The backend is a Node.js application using the Express.js framework, structured using a layered pattern:

- Routes – define the URL paths and HTTP methods exposed by the API and map them to controller functions

- Controllers – contain the request-handling logic: validating input, invoking model methods, and shaping the JSON response
- Models – Mongoose schemas describing the shape of Users, Doctors, and Appointments and providing an interface to MongoDB
- Middleware – cross-cutting logic such as JWT verification (separate middleware for patient, doctor, and admin routes), file upload handling (Multer), and centralized error handling
- Config – initializes the MongoDB connection and the Cloudinary SDK using environment variables

### 3.4 Data Flow Example: Booking an Appointment

1. The patient searches for a doctor and opens the doctor's appointment page in the frontend.
2. The frontend calls GET /api/doctor/list to render available doctors and their time slots.
3. The patient selects a date/time and submits the booking form.
4. The frontend sends a POST request to /api/user/book-appointment with the doctor ID, date, and time slot, including the patient's JWT in the request header.
5. The authUser middleware verifies the token and attaches the patient's ID to the request.
6. The controller checks the doctor's booked slots to avoid double-booking, then creates a new Appointment document in MongoDB.
7. A JSON response confirms the booking; the frontend redirects the patient to their appointments list.
8. If online payment is chosen, a subsequent call initiates a Stripe or Razorpay session, and a webhook/verification step marks the appointment as paid.

### 3.5 Technology Stack Summary

| Layer              | Technology                                        | Purpose                                             |
|--------------------|---------------------------------------------------|-----------------------------------------------------|
| Frontend (Patient) | React.js, React Router, Axios, Context API        | Public-facing UI for booking and account management |
| Frontend (Admin)   | React.js, React Router, Axios                     | Doctor and admin dashboards                         |
| Backend            | Node.js, Express.js                               | REST API and business logic                         |
| Database           | MongoDB, Mongoose                                 | Persistent storage of users, doctors, appointments  |
| Authentication     | JSON Web Tokens (JWT), bcrypt                     | Stateless authentication and password hashing       |
| File Storage       | Cloudinary                                        | Hosting of profile and doctor images                |
| Payments           | Stripe, Razorpay                                  | Online payment processing                           |
| Deployment         | Vercel (frontend), Node-compatible host (backend) | Hosting the production build                        |

## 4. Data Model

---

The application uses three primary MongoDB collections. Field names below reflect the conventions typically used in this style of MERN booking application; refer to the files under backend/models for the authoritative schema in this repository.

### 4.1 User (Patient) Schema

| Field    | Type   | Description                           |
|----------|--------|---------------------------------------|
| name     | String | Patient's full name                   |
| email    | String | Unique login identifier               |
| password | String | bcrypt-hashed password                |
| image    | String | Cloudinary URL of the profile picture |
| address  | Object | Line 1 / Line 2 address information   |
| gender   | String | Patient's gender                      |
| dob      | String | Date of birth                         |
| phone    | String | Contact number                        |

### 4.2 Doctor Schema

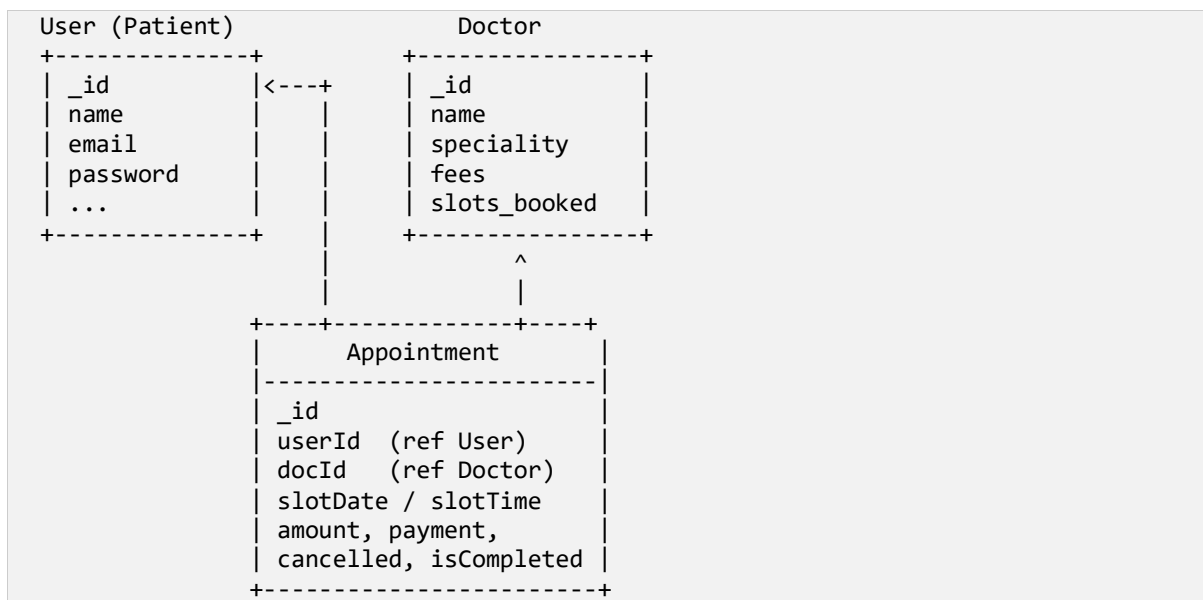
| Field        | Type    | Description                                                |
|--------------|---------|------------------------------------------------------------|
| name         | String  | Doctor's full name                                         |
| email        | String  | Unique login identifier                                    |
| password     | String  | bcrypt-hashed password                                     |
| image        | String  | Cloudinary URL of the doctor's photo                       |
| speciality   | String  | Medical specialty (e.g., General physician, Dermatologist) |
| degree       | String  | Academic qualification                                     |
| experience   | String  | Years of experience                                        |
| about        | String  | Short biography / description                              |
| fees         | Number  | Consultation fee                                           |
| address      | Object  | Clinic/office address                                      |
| available    | Boolean | Whether the doctor is currently accepting bookings         |
| slots_booked | Object  | Map of date to an array of already-booked time slots       |

### 4.3 Appointment Schema

| Field       | Type         | Description                                 |
|-------------|--------------|---------------------------------------------|
| userId      | String (ref) | Reference to the booking patient            |
| docId       | String (ref) | Reference to the booked doctor              |
| slotDate    | String       | Date of the appointment                     |
| slotTime    | String       | Time slot of the appointment                |
| userData    | Object       | Snapshot of patient details at booking time |
| docData     | Object       | Snapshot of doctor details at booking time  |
| amount      | Number       | Consultation fee charged                    |
| date        | Number       | Timestamp the booking was created           |
| cancelled   | Boolean      | Whether the appointment was cancelled       |
| payment     | Boolean      | Whether online payment has been completed   |
| isCompleted | Boolean      | Whether the consultation has taken place    |

## 4.4 Entity Relationship Overview

Figure 4.1 – Relationships between the three core collections



## 5. Setup Instructions

---

### 5.1 Prerequisites

- Node.js (v16 or later) and npm installed
- A MongoDB database – either a local instance or a free MongoDB Atlas cluster
- A Cloudinary account for image uploads (cloud name, API key, API secret)
- A Stripe account and API key for card payments
- A Razorpay account and API keys for UPI/card payments in India
- Git, to clone the repository

### 5.2 Installing Dependencies

The project is split into three independently installable applications.

#### 5.2.1 Backend

```
cd backend
npm install
```

#### 5.2.2 Frontend (Patient Site)

```
cd ../frontend
npm install
```

#### 5.2.3 Admin Panel

```
cd ../admin
npm install
```

### 5.4 Environment Variables

Create a .env file inside the backend directory with the following keys:

```
MONGODB_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret
ADMIN_EMAIL=your_admin_email
ADMIN_PASSWORD=your_admin_password
STRIPE_API_KEY=your_stripe_api_key
RAZORPAY_KEY_ID=your_razorpay_key_id
RAZORPAY_KEY_SECRET=your_razorpay_key_secret
CLOUDINARY_NAME=your_cloudinary_name
CLOUDINARY_API_KEY=your_cloudinary_api_key
CLOUDINARY_SECRET_KEY=your_cloudinary_secret_key
```

If the frontend or admin apps call the backend using an environment-driven base URL, create a corresponding .env in each app, for example:

```
VITE_BACKEND_URL=http://localhost:4000
```

### 5.5 Verifying the Setup

9. Start the backend and confirm the console prints a successful MongoDB connection message.

10. Open the frontend in a browser and confirm the doctor list loads on the Home page.
11. Log into the admin panel using the ADMIN\_EMAIL / ADMIN\_PASSWORD credentials and confirm the dashboard loads.

## 6. Folder Structure

---

### 6.1 Repository Layout

```
BookADoctor/
├── admin/          # React admin & doctor panel
├── backend/        # Node.js / Express REST API
├── frontend/       # React patient-facing site
├── .gitignore
└── README.md
```

### 6.2 Frontend (frontend/)

```
frontend/
├── src/
│   ├── assets/      # Images, icons, static data
│   ├── components/  # Navbar, Footer, DoctorCard, Banner, etc.
│   ├── context/     # AppContext - global state (token, doctors, user)
│   ├── pages/       # Home, Doctors, About, Contact, Appointment,
│   │               # MyProfile, MyAppointments, Login
│   ├── App.jsx
│   └── main.jsx
├── index.html
└── package.json
```

### 6.3 Admin Panel (admin/)

```
admin/
├── src/
│   ├── assets/
│   ├── components/  # Sidebar, Navbar
│   ├── context/     # AdminContext, DoctorContext
│   ├── pages/
│   │   ├── Admin/   # Dashboard, AddDoctor, DoctorsList, AllAppointments
│   │   └── Doctor/  # DoctorDashboard, DoctorAppointments, DoctorProfile
│   ├── App.jsx
│   └── main.jsx
├── index.html
└── package.json
```

### 6.4 Backend (backend/)

```
backend/
├── config/
│   ├── mongodb.js   # Database connection
│   └── cloudinary.js # Cloudinary SDK setup
├── controllers/
│   ├── userController.js
│   ├── doctorController.js
│   └── adminController.js
├── middleware/
│   ├── authUser.js
│   ├── authDoctor.js
│   ├── authAdmin.js
│   └── multer.js     # Image upload handling
└── models/
```

```
├── userModel.js
├── doctorModel.js
├── appointmentModel.js
├── routes/
│   ├── userRoute.js
│   ├── doctorRoute.js
│   └── adminRoute.js
├── server.js          # Express app entry point
└── package.json
```

## 7. Running the Application

---

Each of the three applications runs as a separate process during development.

### 7.1 Start the Backend

```
cd backend
npm run server
# or: npm start
```

The API starts on the port defined in `server.js` (commonly 4000) and connects to MongoDB using `MONGODB_URI`.

### 7.2 Start the Frontend

```
cd frontend
npm run dev
# or: npm start
```

### 7.3 Start the Admin Panel

```
cd admin
npm run dev
# or: npm start
```

### 7.4 Typical Local Ports

| Application | Default Port        | URL                   |
|-------------|---------------------|-----------------------|
| Backend API | 4000                | http://localhost:4000 |
| Frontend    | 5173 (Vite) or 3000 | http://localhost:5173 |
| Admin Panel | 5174 (Vite) or 3001 | http://localhost:5174 |

Exact ports depend on the build tool configuration (Vite vs. Create React App) used in each `package.json`.

## 8. API Documentation

---

All endpoints are prefixed with /api and return JSON. Endpoints under /api/user require a patient JWT, endpoints under /api/doctor (except login and list) require a doctor JWT, and endpoints under /api/admin require an admin JWT. Tokens are issued at login and are expected to be sent on subsequent requests via a request header (commonly named token or Authorization).

### 8.1 User (Patient) Endpoints

#### POST

##### /api/user/register

**Access:** Public

Creates a new patient account. The password is hashed with bcrypt before being stored, and a JWT is returned so the patient is signed in immediately after registering.

**Request Body:**

```
{
  "name": "Jane Doe",
  "email": "jane@example.com",
  "password": "SecurePass123"
}
```

**Example Response:**

```
{
  "success": true,
  "token": "<jwt_token>"
}
```

#### POST

##### /api/user/login

**Access:** Public

Authenticates a patient using email and password and returns a JWT on success.

**Request Body:**

```
{
  "email": "jane@example.com",
  "password": "SecurePass123"
}
```

**Example Response:**

```
{
  "success": true,
  "token": "<jwt_token>"
}
```

#### GET

##### /api/user/get-profile

**Access:** Patient

Returns the profile of the currently authenticated patient.

**Example Response:**

```
{
  "success": true,
  "userData": {
    "name": "Jane Doe",
    "email": "jane@example.com",
    "image": "https://res.cloudinary.com/...",
    "address": { "line1": "", "line2": "" },
    "gender": "Female",
    "dob": "1998-05-14",
    "phone": "9999999999"
  }
}
```

## POST

### /api/user/update-profile

**Access:** Patient

Updates the patient's profile fields and, optionally, the profile picture (multipart form data).

**Request Body:**

```
{
  "name": "Jane Doe",
  "phone": "9999999999",
  "address": "{ \"line1\": \"221B Baker St\", \"line2\": \"\" }",
  "dob": "1998-05-14",
  "gender": "Female"
}
```

**Example Response:**

```
{
  "success": true,
  "message": "Profile Updated"
}
```

## POST

### /api/user/book-appointment

**Access:** Patient

Books an appointment with a doctor for a given date and time slot, after checking that the slot is not already taken.

**Request Body:**

```
{
  "docId": "64f1a2b3c4d5e6f7a8b9c0d1",
  "slotDate": "12_08_2026",
  "slotTime": "10:00 AM"
}
```

**Example Response:**

```
{
  "success": true,
```

```
{
  "message": "Appointment Booked"
}
```

## POST

**/api/user/cancel-appointment**

**Access:** Patient

Cancels an existing appointment belonging to the authenticated patient and frees the doctor's slot.

**Request Body:**

```
{
  "appointmentId": "64f1a2b3c4d5e6f7a8b9c0d2"
}
```

**Example Response:**

```
{
  "success": true,
  "message": "Appointment Cancelled"
}
```

## GET

**/api/user/appointments**

**Access:** Patient

Returns the list of all appointments (past and upcoming) for the authenticated patient.

**Example Response:**

```
{
  "success": true,
  "appointments": [ { "...": "..." } ]
}
```

## POST

**/api/user/payment-stripe**

**Access:** Patient

Creates a Stripe checkout session for a given appointment.

**Request Body:**

```
{
  "appointmentId": "64f1a2b3c4d5e6f7a8b9c0d2"
}
```

**Example Response:**

```
{
  "success": true,
  "session_url": "https://checkout.stripe.com/..."
}
```

## POST

**/api/user/payment-razorpay**

**Access:** Patient

Creates a Razorpay order for a given appointment, to be confirmed client-side via the Razorpay checkout widget.

**Request Body:**

```
{
  "appointmentId": "64f1a2b3c4d5e6f7a8b9c0d2"
}
```

**Example Response:**

```
{
  "success": true,
  "order": { "id": "order_...", "amount": 5000 }
}
```

## 8.2 Doctor Endpoints

### POST

**/api/doctor/login**

**Access:** Public

Authenticates a doctor using email and password and returns a JWT on success.

**Request Body:**

```
{
  "email": "dr.james@example.com",
  "password": "SecurePass123"
}
```

**Example Response:**

```
{
  "success": true,
  "token": "<jwt_token>"
}
```

### GET

**/api/doctor/list**

**Access:** Public

Returns the list of all doctors with public-facing details, for display on the patient site.

**Example Response:**

```
{
  "success": true,
  "doctors": [
    {
      "_id": "64f1a2b3c4d5e6f7a8b9c0d1",
      "name": "Dr. Richard James",
      "speciality": "General physician",
      "degree": "MBBS",
      "experience": "4 Years",
      "fees": 50,
      "available": true
    }
  ]
}
```

```
]
}
```

## GET

### /api/doctor/appointments

**Access:** Doctor

Returns all appointments booked with the authenticated doctor.

#### Example Response:

```
{
  "success": true,
  "appointments": [ { "...": "..." } ]
}
```

## POST

### /api/doctor/complete-appointment

**Access:** Doctor

Marks a specific appointment as completed.

#### Request Body:

```
{
  "appointmentId": "64f1a2b3c4d5e6f7a8b9c0d2"
}
```

#### Example Response:

```
{
  "success": true,
  "message": "Appointment Completed"
}
```

## POST

### /api/doctor/cancel-appointment

**Access:** Doctor

Cancels a specific appointment from the doctor's side.

#### Request Body:

```
{
  "appointmentId": "64f1a2b3c4d5e6f7a8b9c0d2"
}
```

#### Example Response:

```
{
  "success": true,
  "message": "Appointment Cancelled"
}
```

## POST

### /api/doctor/update-profile

**Access:** Doctor

Updates the authenticated doctor's fee, address, description, or availability status.

**Request Body:**

```
{
  "fees": 60,
  "address": "{\\"line1\\":\\"Clinic Rd\\",\\"line2\\":\\"\\\"}",
  "available": true
}
```

**Example Response:**

```
{
  "success": true,
  "message": "Profile Updated"
}
```

## GET

**/api/doctor/dashboard**

**Access:** Doctor

Returns aggregated dashboard statistics for the doctor: earnings, patient count, appointment count, and latest bookings.

**Example Response:**

```
{
  "success": true,
  "dashData": {
    "earnings": 1200,
    "appointments": 24,
    "patients": 18,
    "latestAppointments": [ { "...": "..." } ]
  }
}
```

## 8.3 Admin Endpoints

### POST

**/api/admin/login**

**Access:** Public

Authenticates the platform administrator using credentials configured via environment variables and returns a JWT.

**Request Body:**

```
{
  "email": "admin@prescripto.com",
  "password": "AdminPass123"
}
```

**Example Response:**

```
{
  "success": true,
  "token": "<jwt_token>"
}
```

## POST

**/api/admin/add-doctor**

**Access:** Admin

Creates a new doctor profile, including an image upload (multipart form data).

**Request Body:**

```
{
  "name": "Dr. Emily Larson",
  "email": "emily.larson@example.com",
  "password": "SecurePass123",
  "speciality": "Gynecologist",
  "degree": "MBBS, MD",
  "experience": "6 Years",
  "fees": 55,
  "about": "...",
  "address": "{ \"line1\": \"...\", \"line2\": \"...\" }"
}
```

**Example Response:**

```
{
  "success": true,
  "message": "Doctor Added"
}
```

## GET

**/api/admin/all-doctors**

**Access:** Admin

Returns the full list of registered doctors, including internal fields not exposed on the public endpoint.

**Example Response:**

```
{
  "success": true,
  "doctors": [ { "...": "..." } ]
}
```

## POST

**/api/admin/change-availability**

**Access:** Admin

Toggles a doctor's availability flag.

**Request Body:**

```
{
  "docId": "64f1a2b3c4d5e6f7a8b9c0d1"
}
```

**Example Response:**

```
{
  "success": true,
  "message": "Availability Changed"
}
```

## GET

### /api/admin/appointments

**Access:** Admin

Returns every appointment recorded on the platform.

**Example Response:**

```
{
  "success": true,
  "appointments": [ { "...": "..." } ]
}
```

## POST

### /api/admin/cancel-appointment

**Access:** Admin

Cancels any appointment on the platform, regardless of which patient or doctor it belongs to.

**Request Body:**

```
{
  "appointmentId": "64f1a2b3c4d5e6f7a8b9c0d2"
}
```

**Example Response:**

```
{
  "success": true,
  "message": "Appointment Cancelled"
}
```

## GET

### /api/admin/dashboard

**Access:** Admin

Returns platform-wide statistics: total doctors, total patients, total appointments, and recent bookings.

**Example Response:**

```
{
  "success": true,
  "dashData": {
    "doctors": 12,
    "appointments": 140,
    "patients": 96,
    "latestAppointments": [ { "...": "..." } ]
  }
}
```

## 8.4 Common Error Response

Failed requests generally follow a consistent shape so the frontend can display a uniform error message:

```
{
  "success": false,
  "message": "Not Authorized. Login Again"
}
```

## 8.5 HTTP Status Conventions

| Status | Meaning                                                                           |
|--------|-----------------------------------------------------------------------------------|
| 200    | Request succeeded (the JSON body's success flag indicates business-level outcome) |
| 400    | Malformed request or validation error                                             |
| 401    | Missing or invalid authentication token                                           |
| 404    | Resource not found                                                                |
| 500    | Unhandled server error                                                            |

## 9. Authentication & Authorization

### 9.1 Overview

This uses stateless authentication based on JSON Web Tokens (JWT). Rather than storing session state on the server, each successful login returns a signed token that encodes the user's identity; the client stores this token and includes it with subsequent requests. This approach fits the REST API model well and avoids the need for server-side session storage.

### 9.2 Password Storage

Passwords are never stored in plain text. During registration or doctor/admin account creation, the password is hashed using bcrypt with a salt, and only the resulting hash is persisted in MongoDB. During login, the submitted password is hashed and compared against the stored hash.

### 9.3 Token Issuance

On successful login, the backend signs a JWT using a secret key (JWT\_SECRET) containing the user's document ID (and, for doctors/admins, their role). The token is returned to the client in the JSON response body.

### 9.4 Token Verification Middleware

Three separate middleware functions protect the three categories of routes:

- **authUser:** verifies a patient's token before allowing access to `/api/user/*` protected routes
- **authDoctor:** verifies a doctor's token before allowing access to `/api/doctor/*` protected routes
- **authAdmin:** verifies the admin token before allowing access to `/api/admin/*` protected routes

Each middleware reads the token from the request headers, verifies its signature against JWT\_SECRET, and attaches the decoded identity to the request object so downstream controllers know which user is making the call. If verification fails, the middleware short-circuits the request with a 401-style JSON error rather than allowing it to reach the controller.

### 9.5 Authorization Boundaries

Authorization is enforced by scoping queries to the authenticated identity: a patient can only see and cancel their own appointments, a doctor can only manage appointments where they are the assigned doctor, and only the admin identity (configured via ADMIN\_EMAIL/ADMIN\_PASSWORD rather than a public sign-up flow) can add doctors or act across the entire dataset.

### 9.6 Login Sequence

Figure 9.1 – Simplified login and authenticated request sequence

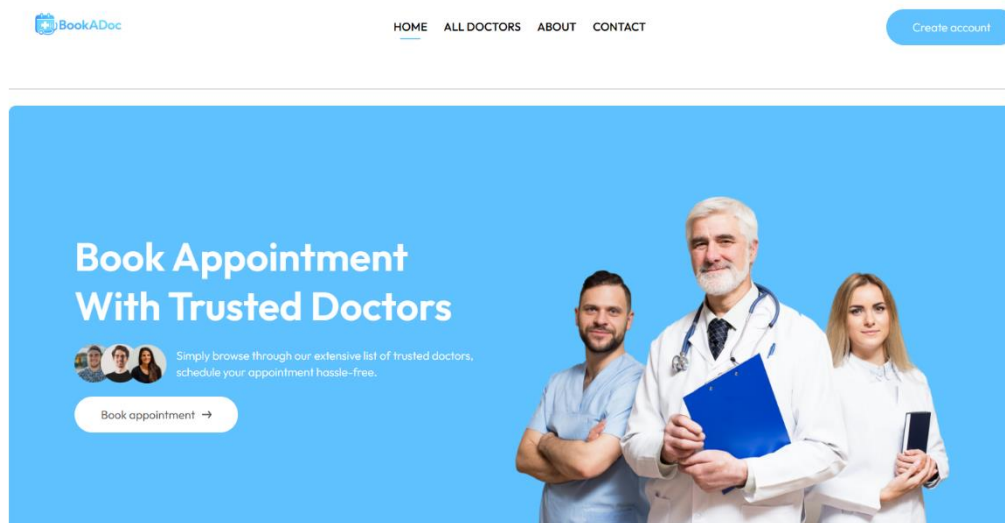




## 10. User Interface

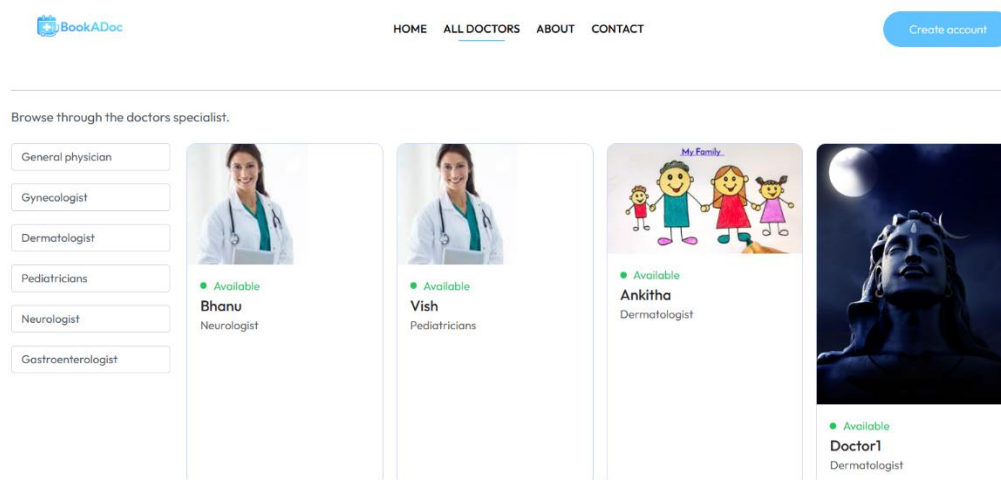
### 10.1 Home Page

The Home page presents a hero banner, a specialty-based quick search, and a curated list of top doctors. A footer links to About, Delivery Information, Privacy Policy, and Get in Touch sections.



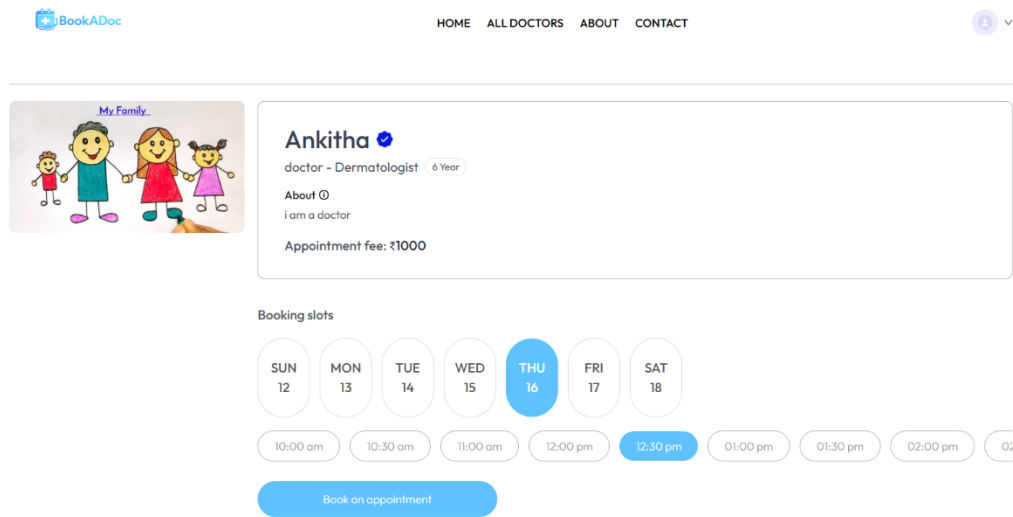
### 10.2 All Doctors Page

Lists every registered doctor with filtering by specialty. Selecting a doctor navigates to their individual appointment page.



### 10.3 Doctor Appointment Page

Shows the doctor's photo, qualification, experience, and description, an appointment booking widget for choosing a date and time slot, a payment and a related-doctors section. Booking requires the patient to be logged in.



The screenshot shows the BookADoc website header with the logo and navigation links: HOME, ALL DOCTORS, ABOUT, and CONTACT. A user profile icon is in the top right. The main content area features a doctor's profile for Ankitha, a Dermatologist with 6 years of experience. To the left of the profile is a placeholder image labeled 'My Family...'. Below the profile, there is a 'Booking slots' section with a calendar view showing days from Sunday to Saturday. The 16th of the month (Thursday) is selected. Below the calendar, time slots are listed from 10:00 am to 02:00 pm, with 12:30 pm being the selected slot. A 'Book on appointment' button is at the bottom.

BookADoc

HOME ALL DOCTORS ABOUT CONTACT

My Family...

**Ankitha**  
doctor - Dermatologist 6 Year

About ⓘ  
i am a doctor

Appointment fee: ₹1000

Booking slots

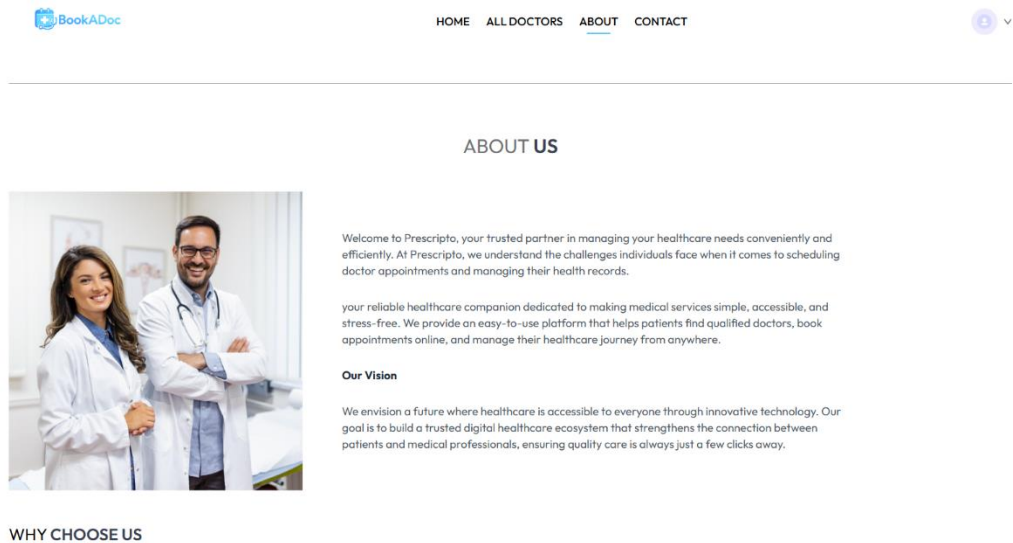
SUN 12 MON 13 TUE 14 WED 15 THU 16 FRI 17 SAT 18

10:00 am 10:30 am 11:00 am 12:00 pm 12:30 pm 01:00 pm 01:30 pm 02:00 pm 02:30 pm

Book on appointment

## 10.4 About Page

Describes Book A Doctor mission and a “Why Choose Us” section covering efficiency, convenience, and personalization.



The screenshot shows the 'About Us' page of the BookADoc website. The header is identical to the previous screenshot. The main heading is 'ABOUT US'. Below it is a large image of a male and female doctor in white coats. To the right of the image is a text block containing a welcome message, a description of the platform's mission, and a section titled 'Our Vision' which describes the goal of building a digital healthcare ecosystem.

BookADoc

HOME ALL DOCTORS ABOUT CONTACT

## ABOUT US



Welcome to Prescripto, your trusted partner in managing your healthcare needs conveniently and efficiently. At Prescripto, we understand the challenges individuals face when it comes to scheduling doctor appointments and managing their health records.

your reliable healthcare companion dedicated to making medical services simple, accessible, and stress-free. We provide an easy-to-use platform that helps patients find qualified doctors, book appointments online, and manage their healthcare journey from anywhere.

**Our Vision**

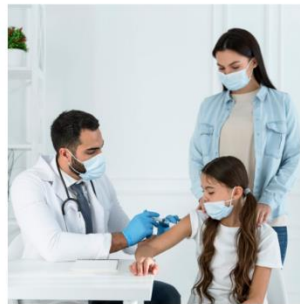
We envision a future where healthcare is accessible to everyone through innovative technology. Our goal is to build a trusted digital healthcare ecosystem that strengthens the connection between patients and medical professionals, ensuring quality care is always just a few clicks away.

WHY CHOOSE US

## 10.5 Contact Page

Displays office address, contact details, and information about job opportunities.

## CONTACT US



### OUR OFFICE

Hyderabad  
India

Tel: +91 1234567890  
Email: bookadoctor@gmail.com

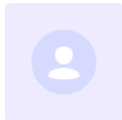
### CAREERS AT PRESCRIPTO

Learn more about our teams and job openings.

[Explore Jobs](#)

## 10.6 My Profile Page

Allows a logged-in patient to view and edit personal information, upload a profile photo, and review their appointment history, with a logout option.



### Alex

#### CONTACT INFORMATION

Email id: alexa@gmail.com  
Phone: 0000000000  
Address:

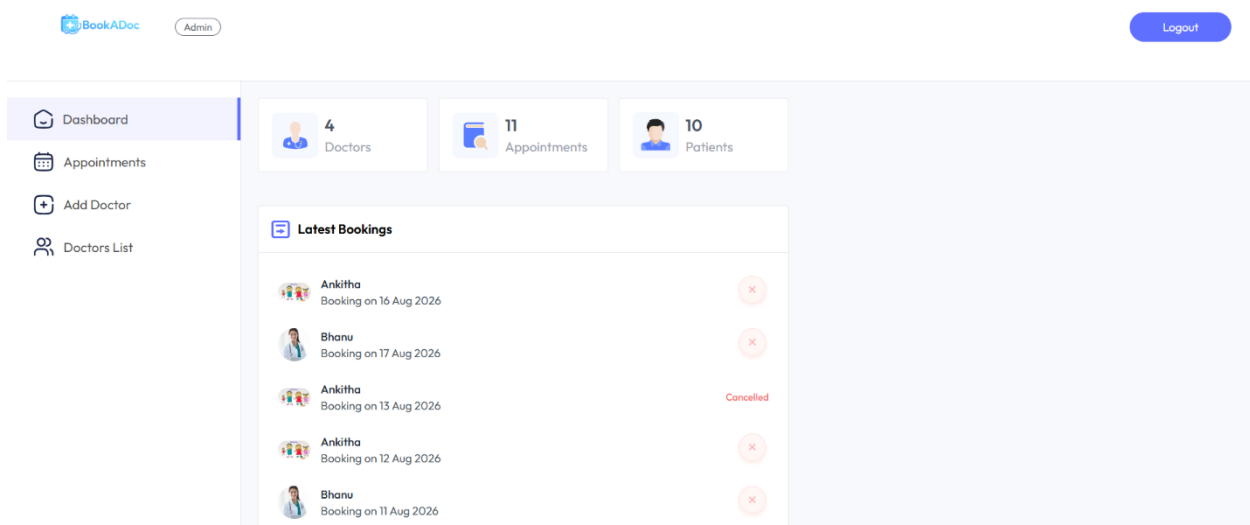
#### BASIC INFORMATION

Gender: Not Selected  
Birthday: Not Selected

[Edit](#)

## 10.7 Admin Dashboard

Displays platform-wide statistics (doctor count, appointment count, patient count, recent bookings) with the ability to cancel bookings directly from the dashboard.



## 10.8 Add Doctor Page (Admin)

A form for entering a new doctor's image, name, email, password, specialty, degree, experience, fees, address, and description.

The screenshot shows the BookADoc Admin Dashboard with the 'Add Doctor' form selected in the sidebar. The form is titled 'Add Doctor' and includes a section for 'Upload doctor picture'. Below this, there are several input fields and dropdown menus for doctor information: Your name (Name), Doctor Email (Email), Set Password (Password), Experience (1 Year), Fees, Speciality (General physician), Degree (Degree), Address (Address 1, Address 2).

| Field        | Value                |
|--------------|----------------------|
| Your name    | Name                 |
| Doctor Email | Email                |
| Set Password | Password             |
| Experience   | 1 Year               |
| Fees         |                      |
| Speciality   | General physician    |
| Degree       | Degree               |
| Address      | Address 1, Address 2 |



- Dashboard
- Appointments
- Add Doctor
- Doctors List

## All Appointments

| # | Patient                                                                                          | Age | Date & Time           | Doctor                                                                                     | Fees  | Action                                                                              |
|---|--------------------------------------------------------------------------------------------------|-----|-----------------------|--------------------------------------------------------------------------------------------|-------|-------------------------------------------------------------------------------------|
| 1 |  Robin          | NaN | 16 Aug 2026, 11:30 AM |  Ankitha | 1000  |  |
| 2 |  Alex           | NaN | 17 Aug 2026, 12:00 PM |  Bhanu   | 19997 |  |
| 3 |  Alice          | NaN | 13 Aug 2026, 10:30 AM |  Ankitha | 1000  | Cancelled                                                                           |
| 4 |  Alice          | NaN | 12 Aug 2026, 11:00 AM |  Ankitha | 1000  |  |
| 5 |  John Deo       | NaN | 11 Aug 2026, 12:00 PM |  Bhanu   | 19997 |  |
| 6 |  Ankitha Komati | NaN | 11 Aug 2026, 11:30 AM |  Bhanu   | 19997 |  |
| 7 |  Komati Ankitha | NaN | 11 Aug 2026, 11:00 AM |  Bhanu   | 19997 |  |
| 8 |  John           | NaN | 11 Aug 2026, 10:00 AM |  Bhanu   | 19997 |  |

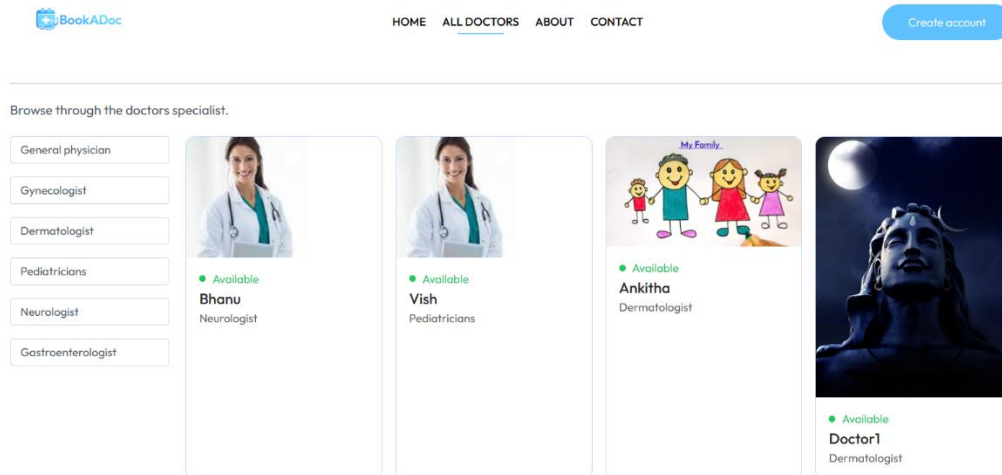
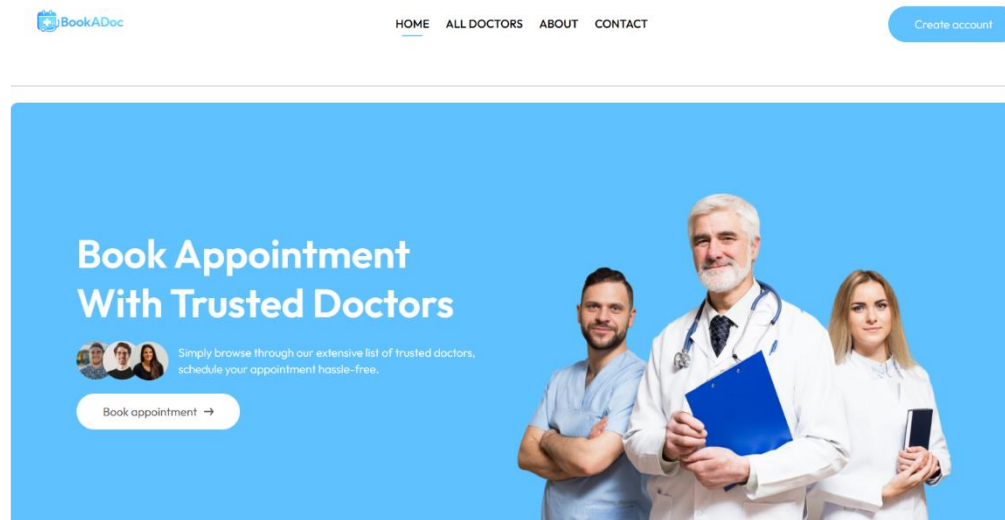
## 11. Testing

---

### 11.1 Manual Test Coverage

| Area             | Scenario                                         | Expected Result                                             |
|------------------|--------------------------------------------------|-------------------------------------------------------------|
| Registration     | Patient signs up with a new email                | Account created, JWT returned, patient redirected to Home   |
| Registration     | Patient signs up with an already-used email      | Registration blocked with a clear error message             |
| Login            | Correct credentials for patient / doctor / admin | JWT returned, redirected to the appropriate dashboard       |
| Login            | Incorrect password                               | Login rejected with an error message                        |
| Booking          | Book an open slot                                | Appointment created and visible under My Appointments       |
| Booking          | Attempt to book an already-taken slot            | Booking blocked, slot unavailable in the UI                 |
| Cancellation     | Patient cancels an upcoming appointment          | Appointment marked cancelled, slot freed for other patients |
| Payment          | Complete a Stripe/Razorpay test payment          | Appointment payment flag set to true                        |
| Doctor dashboard | Doctor marks an appointment complete             | isCompleted flag set; reflected in earnings                 |
| Admin            | Admin adds a new doctor                          | Doctor appears in All Doctors and on the patient site       |
| Authorization    | Patient token used on an /api/admin/* route      | Request rejected as unauthorized                            |

## 12. Screenshots or Demo



**Ankitha**

doctor - Dermatologist 6 Year

About

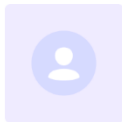
i am a doctor

Appointment fee: ₹1000

## Booking slots

|           |           |           |           |           |           |           |
|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| SUN<br>12 | MON<br>13 | TUE<br>14 | WED<br>15 | THU<br>16 | FRI<br>17 | SAT<br>18 |
| 10:00 am  | 10:30 am  | 11:00 am  | 12:00 pm  | 12:30 pm  | 01:00 pm  | 01:30 pm  |
| 02:00 pm  | 02:30 pm  | 03:00 pm  | 03:30 pm  | 04:00 pm  | 04:30 pm  | 05:00 pm  |

Book an appointment

**Alex**

## CONTACT INFORMATION

Email id: alexa@gmail.com

Phone: 0000000000

Address:

## BASIC INFORMATION

Gender: Not Selected

Birthday: Not Selected

[Edit](#)

### Admin Login

Email

Password

Login

Doctor Login? [Click here](#)



Admin

Logout

- Dashboard
- Appointments
- Add Doctor
- Doctors List

#### Add Doctor



Upload doctor picture

Your name

Name

Speciality

General physician

Doctor Email

Email

Degree

Degree

Set Password

Password

Address

Address 1

Address 2

Experience

1 Year

Fees

## Add Doctor

|                                                                        |                                                                                 |
|------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <div> <div> Email </div> </div>                                        | <div> <div> Degree </div> </div>                                                |
| <div> <div> Set Password </div> <div> Password </div> </div>           | <div> <div> Address </div> <div> Address 1 </div> <div> Address 2 </div> </div> |
| <div> <div> Experience </div> <div> 1 Year </div> </div>               |                                                                                 |
| <div> <div> Fees </div> <div> Doctor fees </div> </div>                |                                                                                 |
| <div> <div> About Doctor </div> <div> write about doctor </div> </div> |                                                                                 |

[Add doctor](#)

## All Doctors



**Bhanu**  
Neurologist  
 Available



**Vish**  
Pediatricians  
 Available



**Ankitha**  
Dermatologist  
 Available



**Doctor1**  
Dermatologist  
 Available

**Albert**  
General physician  
 Available

- Dashboard
- Appointments
- Add Doctor
- Doctors List

## All Appointments

| # | Patient                                                                                          | Age | Date & Time           | Doctor                                                                                     | Fees  | Action                                                                              |
|---|--------------------------------------------------------------------------------------------------|-----|-----------------------|--------------------------------------------------------------------------------------------|-------|-------------------------------------------------------------------------------------|
| 1 |  Robin          | NaN | 16 Aug 2026, 11:30 AM |  Ankitha | 1000  |  |
| 2 |  Alex           | NaN | 17 Aug 2026, 12:00 PM |  Bhanu   | 19997 |  |
| 3 |  Alice          | NaN | 13 Aug 2026, 10:30 AM |  Ankitha | 1000  | Cancelled                                                                           |
| 4 |  Alice          | NaN | 12 Aug 2026, 11:00 AM |  Ankitha | 1000  |  |
| 5 |  John Deo       | NaN | 11 Aug 2026, 12:00 PM |  Bhanu   | 19997 |  |
| 6 |  Ankitha Komati | NaN | 11 Aug 2026, 11:30 AM |  Bhanu   | 19997 |  |
| 7 |  Komati Ankitha | NaN | 11 Aug 2026, 11:00 AM |  Bhanu   | 19997 |  |
| 8 |  John           | NaN | 11 Aug 2026, 10:00 AM |  Bhanu   | 19997 |  |

## 13. Known Issues

---

- No automated test coverage is currently included in the repository
- Environment variable names may need adjustment depending on the deployment platform (e.g., Vercel serverless functions vs. a traditional Node host)
- Payment gateway keys must be kept out of version control; confirm `.env` is listed in `.gitignore` before pushing changes
- Concurrent booking of the same slot by two patients relies on a check-then-write pattern in the controller, which could be tightened with a database-level uniqueness constraint or transaction
- No password-reset / forgot-password flow is documented
- Doctor image uploads depend on Cloudinary; the app will not function correctly without valid Cloudinary credentials configured

## 14. Future Enhancements

---

### 14.1 Functional Enhancements

- Automated appointment reminders via email or SMS
- Doctor ratings and patient reviews
- Video consultation / telemedicine support
- Digital prescriptions generated after a completed appointment
- Multi-language support for broader accessibility
- Forgot-password / email verification flows

### 14.2 Technical Enhancements

- Add an automated test suite (unit, integration, and end-to-end) and a CI pipeline
- Introduce server-side pagination for large doctor and appointment lists
- Add rate limiting and stronger input validation on the API
- Move to refresh-token-based authentication to reduce the exposure window of long-lived JWTs
- Add structured logging and monitoring (e.g., Winston, Sentry) for production observability

### 14.3 Analytics Enhancements

- Revenue trend charts on the admin dashboard
- Patient demographic breakdowns
- Doctor-level performance and utilization reports

